

Agent Concepts and MCP Basics

Fabiana Alves Ferreira

What is an agent

An agent is software capable of acting, making decisions, and executing tasks autonomously. Unlike a regular program, an agent uses a language model (LLM) that follows a continuous cycle of perception, reasoning, and action: it assesses the situation, decides which tools to use, executes, observes the result, and decides the next step — without that path having been rigidly defined in advance.

The core difference between a workflow and an agent is who decides the next steps. In a workflow, the developer defines the order of steps before execution — the sequence is fixed, and the result tends to be the same for the same input. In an agent, the system itself has the autonomy to decide, during execution, how to solve the task — it can skip steps, choose different tools, or change strategy based on what it finds along the way.

My FL-04 pipeline (Draft → Critique → Revise → Human Check) is a workflow, not an agent. I decided that order myself, when writing the structured prompt — the system always follows the four steps in the same sequence, never skipping any. Throughout the real test runs, I had to repeat executions several times, but that happened because of errors (mine or the system's, like when it answered with general knowledge instead of flagging that the information wasn't in the documents), never because the system decided on its own to do something different from what I had structured. It never had the autonomy to choose to skip a step or invent a new one — it only ever executed what I had already designed.

What is MCP

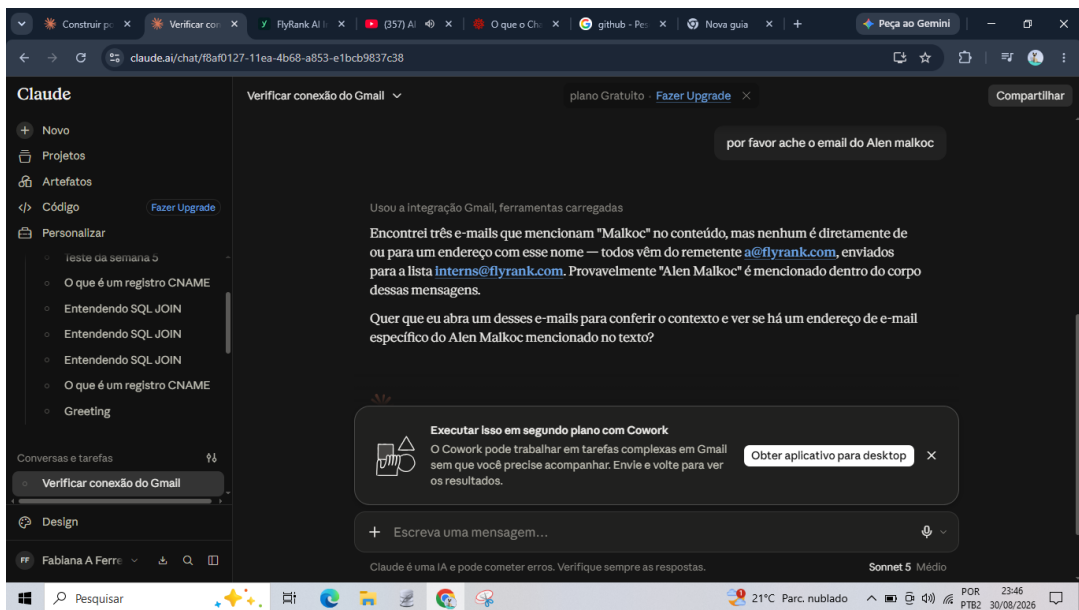
MCP (Model Context Protocol) is an open standard that lets AI applications, like Claude, connect to external data sources and tools — local files, databases, email, calendars, code repositories. It works like a “USB-C port” for AI: instead of every tool needing a custom integration, MCP standardizes that connection.

The protocol is organized around three basic elements. **Tools** are actions the model can execute that have a real effect in the world — running a command, creating or editing a file, calling an external API like GitHub; they're like the model's “arms and legs,” since on its own it can only generate text. **Resources** are sources of information, static or dynamic, that the model consults to understand context before responding or acting — it doesn't alter these resources, only reads them. **Prompts** are pre-configured behavior templates, provided by the MCP server, that guide how the AI should act on specific tasks — ready-made shortcuts for recurring situations.

Testing a real MCP connector

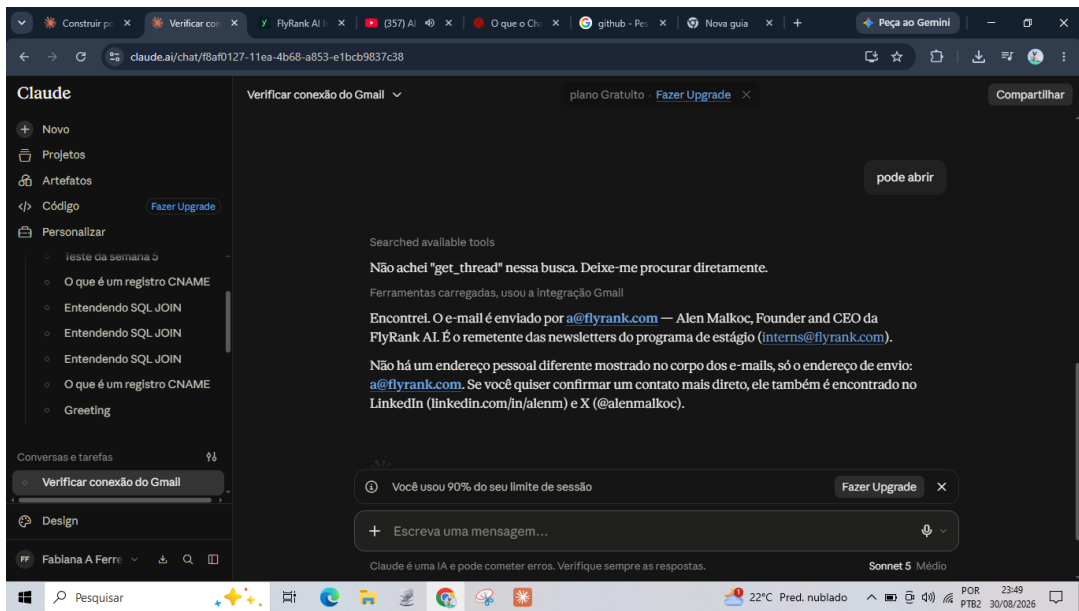
I connected Gmail to Claude through claude.ai's own Connectors (which run on MCP) and executed three tasks that chat alone, without that access, could never do.

Task 1: I asked it to search for emails mentioning “Alen Malkoc.” Claude used the real Gmail integration and found three real emails in my inbox mentioning that name, identifying the common sender and the distribution list.



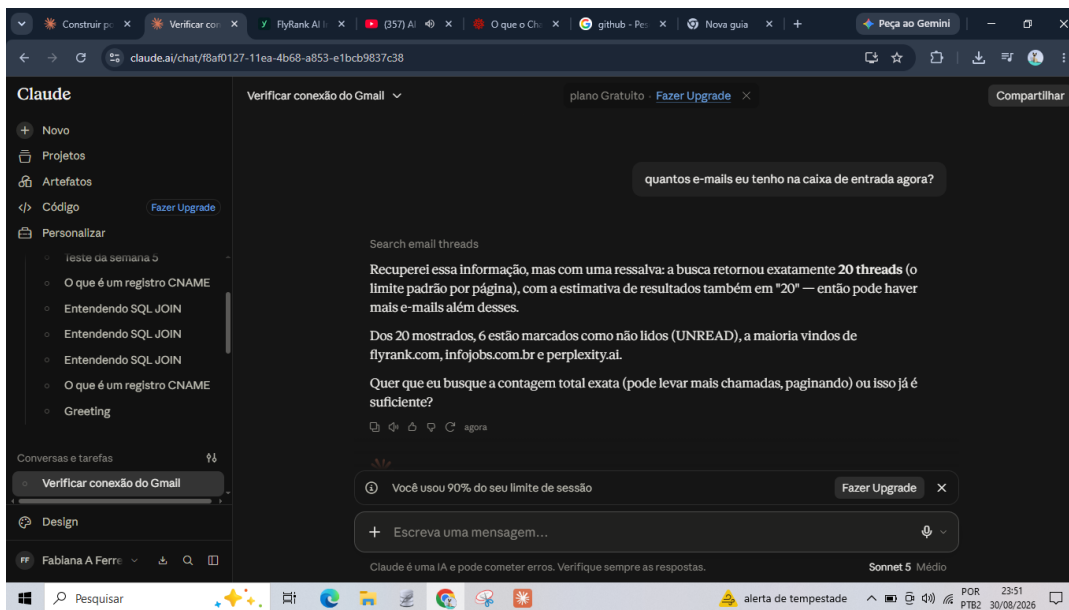
Proof: “Used the Gmail integration, tools loaded,” with a real result from the inbox.

Task 2: I asked it to open one of those emails and confirm the context. The system tried to use a specific tool that didn't exist (“get_thread”), didn't find it, and adapted by searching a different way — confirming that Alen Malkoc is the Founder and CEO of FlyRank AI, with cross-validation on LinkedIn and X.



Proof: a tool attempt that failed, followed by a real adaptation using a different search.

Task 3: I asked for the inbox email count. The system used the email search tool, returned the count (20 visible threads, with an honest note that the search has a pagination limit and there could be more), and identified that 6 were marked unread, with the source of each.



Proof: use of the search tool with real data and a limitation acknowledged by the system itself.

In all three, the answer didn't come from the model's generic knowledge — it came from real data in my account, fetched on the spot, with confirmed real tool use at every step.

What FL-04 would need to become an agent

Today, my draft/critique/revise pipeline is fixed: I decide the order, and the system just executes. To become a real agent, it would need the autonomy to decide, on its own, when each step is necessary — for example, skipping the Critique step when the draft is already good, or repeating the Revise step more than once if the previous fix didn't solve the problem, without me having to ask again. It would also need access to real tools via MCP to fetch the relevant source documents from my Google Drive on its own, instead of me having to paste the raw fact in manually every time. And it would need long-term memory, to remember tone and style decisions I've already approved in previous runs, without me having to repeat the context every new conversation.

Site: <https://fabiana-alves-ferreira.netlify.app>